

Practical - Week - 3

Task - 1

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process States at different stages of execution

```
root@Shamikahasini: ~  Windows PowerShell  root@Shamikahasini: ~  +  v
GNU nano 7.2 process_state.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

void display_info(const char *stage)
{
    printf("\n--- %s ---\n", stage);
    printf("PID : %d\n", getpid());
    printf("PPID : %d\n", getppid());
    fflush(stdout);
}

int main()
{
    pid_t pid;

    // Initial state - before fork()
    display_info("Before fork()");

    // Create child process
    pid = fork();

    if (pid < 0)
    {

root@Shamikahasini:~# nano process_state.c
root@Shamikahasini:~# gcc process_state.c -o process_state
root@Shamikahasini:~# ./process_state

--- Before fork() ---
PID : 1799
PPID : 326

--- Parent: Running ---
PID : 1799
PPID : 326
Parent: Child PID = 1800
Parent: Waiting for child...

--- Child: Running ---
PID : 1800
PPID : 1799
Child: Going to sleep for 5 seconds...

--- Child: After sleep ---
PID : 1800
PPID : 1799
Child: Exiting...

--- Parent: Child completed ---
PID : 1799
PPID : 326
Parent: Exiting...
root@Shamikahasini:~#
```

Observation: The program was executed successfully using the fork() system call. A parent process with PID 1146 created a child process with PID 1147. The child's PPID was 1146, confirming the parent-child relationship. The parent waited while the child executed, and after the child completed its execution, the parent process also terminated.

Result: The experiment successfully demonstrated process creation, PID and PPID identification, parent-child execution, and process termination using fork() and wait().

Task - 2

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as PS, top, and /proc. Document the observations.

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Process started.\n");
    printf("PID = %d\n", getpid());
    fflush(stdout);

    printf("Process is going to sleep for 30 seconds...\n");
    fflush(stdout);

    sleep(30);

    printf("Process woke up and is running.\n");
    fflush(stdout);

    printf("Process will now perform CPU work for 30 seconds...\n");
    fflush(stdout);

    for (volatile unsigned long long i = 0;
         i < 6000000000ULL;
         i++);

    printf("Process completed.\n");
    fflush(stdout);
}

root@Shamikahasini:~# nano state_test.c
root@Shamikahasini:~# gcc state_test.c -o state_test
root@Shamikahasini:~# ./state_test &
[1] 2278
root@Shamikahasini:~# Process started.
PID = 2278
Process is going to sleep for 30 seconds...
Process woke up and is running.
Process will now perform CPU work for 30 seconds...
Process completed.
ps -o pid,ppid,state,sps -o pid,ppid,state,stat,cmd -p 4521
PID      PPID S  STAT CMD
[1]+  Done                  ./state_test
root@Shamikahasini:~# ps -o pid,ppid,state,stat,cmd -p 4521
PID      PPID S  STAT CMD
root@Shamikahasini:~# |
```

Observation :

During the experiment in Ubuntu, the process state transitions were successfully observed using ps, top, and /proc. When the process was executing, its state was shown as **R (Running/Runnable)**. When the process executed sleep(), its state changed to **S (Sleeping/Waiting)**. After the sleep period, the process returned to the **R (Running/Runnable)** state and performed CPU-intensive work. Finally, when the program completed, the process was **terminated**, and its entry was removed from /proc. Thus, the experiment demonstrated the process life cycle:
Ready/Running → Waiting → Ready/Running → Terminated.